

Fiche TP — Semaine 2

Squelette HTML du MiniForum

Développement web, classe passerelle

Durée
indicative : 4h

Construire la page d'accueil du **MiniForum** en HTML pur : un en-tête, une liste de sujets codée en dur, un formulaire d'ajout de message et un pied de page. **Aucun style** pour l'instant — c'est voulu, le CSS viendra en semaine 3.

Ce TP suppose que les notions du polycopié *S02_polycop* (structure HTML5, éléments sémantiques, formulaires) ont été vues en cours. Garde-le à portée de main.
Tu travailleras dans le dépôt miniforum créé au TP de la semaine 1.

1 Reprendre le projet

1. Ouvrir le dossier miniforum dans VS Code (*File* → *Open Folder*).
2. Ouvrir un terminal intégré (*Terminal* → *New Terminal*) et récupérer les éventuelles mises à jour :

```
1 git pull
```

3. Ouvrir le fichier `index.html` créé la semaine dernière. Nous allons **remplacer** son contenu par la vraie structure du forum.

Clique droit sur `index.html` → *Open with Live Server* (extension installée en semaine 1). Ta page s'ouvre dans le navigateur et se **recharge automatiquement** à chaque sauvegarde. Garde-la ouverte pendant tout le TP.

2 Poser le squelette sémantique

On commence par la structure d'ensemble, sans contenu détaillé. L'idée est de découper la page en zones sémantiques avant de les remplir.

1. Remplacer le contenu de `index.html` par ce squelette :

Listing 1: `index.html` — structure d'ensemble

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>MiniForum</title>
7 </head>
8 <body>
9   <header>
```

```

10     <!-- Titre du site + navigation -->
11 </header>
12
13 <main>
14     <!-- Liste des sujets + formulaire -->
15 </main>
16
17 <footer>
18     <!-- Pied de page -->
19 </footer>
20 </body>
21 </html>

```

2. Sauvegarder (Ctrl+S) et vérifier dans le navigateur : la page est **vide** pour l'instant, c'est normal. Seul l'onglet affiche « MiniForum ».

Pourquoi une page vide ?

Les balises `<header>`, `<main>` et `<footer>` ne produisent rien de visible par elles-mêmes : elles **structurent** le document. C'est le contenu qu'on va y placer qui s'affichera.

3 Remplir l'en-tête

Dans le `<header>`, on place le titre du site et une barre de navigation. Voici le **rendu attendu** — à toi d'écrire le HTML qui le produit.



Un `<header>` contenant : un titre `<h1>` « MiniForum », puis un `<nav>` avec une liste `<ul>` de deux liens — « Accueil » (vers `index.html`) et « Nouveau sujet » (vers `#nouveau-sujet`).

Le lien `href="#nouveau-sujet"` est un **lien interne** : il pointera vers l'élément de la page dont l'id est `nouveau-sujet` (notre formulaire, plus bas). Clique dessus une fois le formulaire créé : la page défilera jusqu'à lui.

4 La liste des sujets

C'est le cœur de la page d'accueil. Chaque sujet du forum est un `<article>` : rappelle-toi, un article a du sens tout seul, sorti de son contexte.

Dans le `<main>`, tu vas ajouter une première `<section>` contenant deux ou trois sujets codés en dur. Voici le **rendu attendu** :

Sujets récents `<h2>`

Bienvenue sur le MiniForum ! `<article>`
 Posté par **Alice** — Catégorie : Général
 Présentez-vous dans ce premier sujet.

Comment installer Node.js ? `<article>`
 Posté par **Bob** — Catégorie : Aide
 J'ai un souci avec l'installation sous Windows...

... ton troisième sujet, à toi de le composer `<article>`

`<section>` -- englobe les trois `<article>`

Une `<section>` avec un titre `<h2>` « Sujets récents », puis **trois** `<article>`. Dans chaque article : un titre `<h3>`, une ligne d'information (auteur en `<strong>` et catégorie), et un paragraphe `<p>` de contenu.

« Codé en dur » — pourquoi ?

Ces sujets sont écrits directement dans le HTML : ils ne viennent d'aucune base de données. À ce stade, si tu rafraîchis ou relances la page, ce sont **toujours les mêmes**. C'est la limite volontaire de la phase 1 : elle rend évident, plus tard, le besoin d'une vraie base de données.

5 Le formulaire d'ajout de message

Sous la liste des sujets, toujours dans le `<main>`, on ajoute une seconde `<section>` contenant le formulaire de création de sujet. Cette section portera l'attribut `id="nouveau-sujet"` (la cible du lien de navigation). Voici le **rendu attendu** :

Créer un nouveau sujet `<h2>`

Votre pseudo : `<label> + <input>`
 ex : *alice*

Titre du sujet :

Catégorie : `<select>`
 Général ▼

Votre message : `<textarea>`

Publier `<button type="submit">`

`<section id="nouveau-sujet"> ... <form> ... </form> </section>`

Une `<section id="nouveau-sujet">` avec un `<h2>` et un `<form>` contenant, chacun avec son `<label>` : un champ **pseudo** (text, avec placeholder et required), un champ **titre** (text, required), une liste déroulante **catégorie** (`<select>` avec les options Général / Aide / Projets), une zone **message** (`<textarea>`, maxlength et required), et un `<button type="submit">` « Publier ».

Le formulaire ne « part » nulle part

Clique sur **Publier** : rien ne se passe (ou la page se recharge). C'est normal — il manque le serveur qui recevra les données, prévu pour la **phase 3**. Vérifie en revanche que la **validation HTML5** fonctionne : laisse un champ required vide et clique sur Publier, le navigateur doit bloquer l'envoi.

6 Le pied de page

MiniForum — Projet du cours de développement web
Classe passerelle — cfpt-i

`<footer>`

Un `<footer>` avec deux paragraphes `<p>` : le nom du projet, et la mention de la classe.

7 Valider et sauvegarder

Valider le HTML

1. Aller sur validator.w3.org, onglet *Validate by Direct Input*.
2. Copier-coller tout le contenu de `index.html`, puis cliquer sur *Check*.
3. Corriger les éventuelles erreurs signalées (balise mal fermée, alt manquant...) jusqu'à obtenir un résultat **sans erreur**.

Les plus courantes : une balise ouverte mais jamais fermée, un id utilisé deux fois sur la même page, ou un attribut `for` de `<label>` qui ne correspond à aucun id. Le validateur indique la ligne : lis le message, il est souvent explicite.

Sauvegarder sur GitHub

Dans la continuité du TP de la semaine 1, on enregistre le travail et on l'envoie sur GitHub.

```
1 git add .
2 git commit -m "Squelette HTML du MiniForum : sujets et formulaire"
3 git push
```

Prends l'habitude de committer à chaque étape terminée, avec un message clair, et de pousser en fin de séance. Rafraîchis la page de ton dépôt sur `github.com` : ton `index.html` mis à jour doit y apparaître.

8 Pour aller plus loin (optionnel)

Si tu as terminé en avance, voici de quoi enrichir ta page :

- Ajouter une image de logo dans le `<header>` avec la balise `<img>` (sans oublier l'attribut `alt`).
- Transformer la liste des catégories en un vrai `<nav>` secondaire.
- Ajouter un champ `type="email"` au formulaire et tester la validation automatique du navigateur.
- Ajouter un attribut `maxlength` au champ « titre » pour limiter sa longueur.

9 Points de contrôle

- ☐ La page a une structure sémantique : `<header>`, `<main>`, `<footer>`
- ☐ L'en-tête contient un `<h1>` et une navigation `<nav>`
- ☐ Le `<main>` affiche au moins **trois** sujets, chacun dans un `<article>`
- ☐ Le formulaire contient pseudo, titre, catégorie et message, avec leurs `<label>`
- ☐ Chaque `<label>` est relié à son champ (`for` / `id`)
- ☐ Les champs obligatoires ont l'attribut `required` (validation testée)
- ☐ La page passe `validator.w3.org` **sans erreur**
- ☐ Le travail est poussé sur GitHub avec un message de commit clair

À rendre avant la semaine 3

Le dépôt GitHub `miniforum` mis à jour, avec le squelette HTML validé. La semaine 3 introduira le **CSS** pour donner enfin du style à cette structure.